

JavaScript The Basics

Prepared by V.Anitha Moses

Unit-II

- **CLIENT SIDE PROGRAMMING**
- Java Script: An introduction to JavaScript– JavaScript DOM Model-Date and Objects,- Regular Expressions- Exception Handling- Validation-Built-in objects-Event Handling-DHTML with JavaScript- JSON introduction – Syntax – Function Files – Http Request – SQL.
-

- Deitel and Deitel and Neito, "Internet and World wide web-How to Program", Prentice Hall, 5th Edition, 2011

Introduction

- JavaScript is a scripting language most often used for client-side web development.
- JavaScript is an implementation of the ECMAScript standard.
 - The ECMAScript only defines the syntax/characteristics of the language and a basic set of commonly used objects such as Number, Date, Regular Expression, etc.
- The JavaScript supported in the browsers typically support additional objects.
 - e.g., Window, Frame, Form, DOM object, etc.

ECMAScript

- The responsibility for the development of a scripting standard has been transferred to an international body called the European Computer Manufacturers Association (ECMA).
- The standard developed by the ECMA is called ECMAScript, though browsers still refer to it as JavaScript.
- The latest version is ECMA-262, which is supported by the major browsers.

History

- JavaScript created by Netscape
- JScript created by Microsoft
- IE and Netscape renderings are slightly different

JavaScript / JScript

- Different brands or/and different versions of browsers may support different implementation of JavaScript.
 - They are not fully compatible
- JScript is the Microsoft version of JavaScript.

Introduction to JavaScript

- **JavaScript** is an interpreted programming or script language from Netscape.
- JavaScript is used in Web site development to such things as:
 - automatically change a formatted date on a Web page
 - cause a linked-to-page to appear in a popup window
 - cause text or a graphic image to change during a mouse rollover

Java vs. JavaScript

- Requires the JDK to create the applet
- Requires a Java virtual machine to run the applet
- Applet files are distinct from the XHTML code
- Source code is hidden from the user
- Programs must be saved as separate files and compiled before they can be run
- Programs run on the server side
- Requires a text editor
- Required a browser that can interpret JavaScript code
- JavaScript can be placed within HTML and XHTML
- Source code is made accessible to the user
- Programs cannot write content to the hard disk
- Programs run on the client side

Other Client-side Languages

- Internet Explorer supports JScript.
- JScript is identical to JavaScript, but there are some JavaScript commands not supported in JScript, and vice versa.
- Other client-side programming languages are also available to Web page designers, such as the Internet Explorer scripting language, VBScript.

What can we do with JavaScript?

- To create interactive user interface in a web page (e.g., menu, pop-up alert, windows, etc.)
- Manipulating web content dynamically
 - Change the content and style of an element
 - Replace images on a page without page reload
 - Hide/Show contents
- Generate HTML contents on the fly
- Form validation
- AJAX (e.g. Google complete)
- etc.

General Format

```
<!doctype ...>
<html>
<Head>
<Title> Name of web page </title>
<script type="text/javascript">
...script goes here
</script>
</head>
<body>
...page body here: text, forms, tables
...more JavaScript if needed
...onload, onclick, etc. commands here
</body>
</html>
```

Characteristics

- Case sensitive
- Object oriented
- Produces an HTML document
- Dynamically typed
- Standard operator precedence
- Overloaded operators
- Reserved words

Using the `<script>` Tag

- To embed a client-side script in a Web page, use the element:

`<script type="text/javascript" >`

script commands and comments

`</script>`

- To access an external script, use:

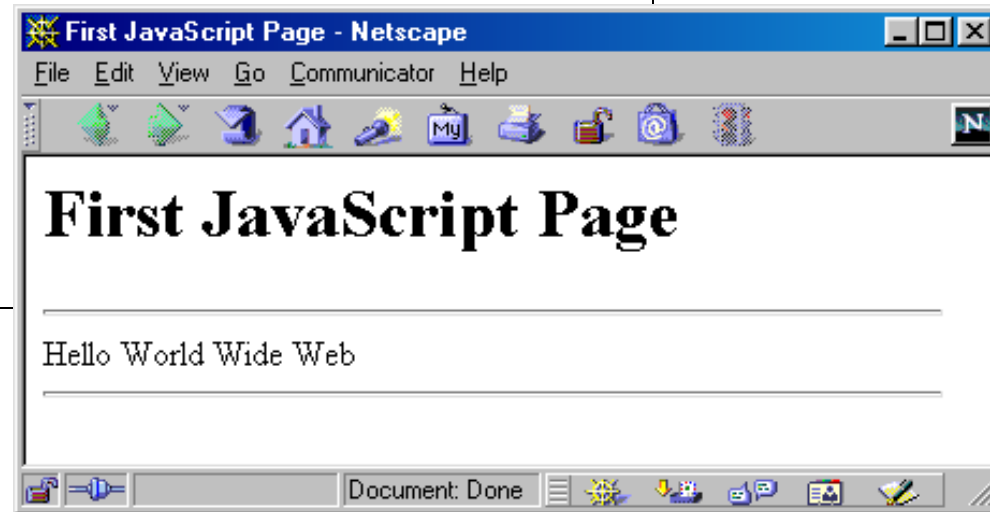
`<script src="url" type="text/javascript">`

script commands and comments

`</script>`

A Simple Script

```
<html>
<head><title>First JavaScript Page</title></head>
<body>
<h1>First JavaScript Page</h1>
<script type="text/javascript">
    document.write("<hr>");
    document.write("Hello World Wide Web");
    document.write("<hr>");
</script>
</body>
</html>
```



Embedding JavaScript

```
<html>
<head><title>First JavaScript Program</title></head>
<body>
<script type="text/javascript"
      src="your_source_file.js"></script>
</body>
</html>
```

Inside your_source_file.js

```
document.write("<hr>");
document.write("Hello World Wide Web");
document.write("<hr>");
```

- Use the **src** attribute to include JavaScript codes from an external file.
- The included code is inserted in place.

Embedding JavaScript

- The scripts inside an HTML document is interpreted in the order they appear in the document.
 - Scripts in a function is interpreted when the function is called.
- So where you place the `<script>` tag matters.

document.writeln

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML>
<!-- Welcome to JavaScript -->
<HEAD>
<TITLE> Welcome to JavaScript </TITLE>
<SCRIPT TYPE="text/javascript">
    document.writeln( "<FONT COLOR='magenta'><H1>Welcome to ",
        "JavaScript Programming!</H1></FONT>" );
</SCRIPT>
</HEAD>
<BODY>
</BODY>
</HTML>
```

document.write

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML>
<HEAD>
<TITLE> Using document.write </TITLE>
<SCRIPT TYPE="text/javascript">
    document.write ( "<H1>Welcome to ");
    document.writeln( "JavaScript Programming!</H1>" );
</SCRIPT>
</HEAD>
<BODY>
</BODY>
</HTML>
```

JavaScript Statements

```
<html>  
<head><title>My Page</title></head>  
<body>  
<script language="JavaScript">
```

```
document.write('This is my first →  
JavaScript Page');
```

```
</script>
```

```
</body>
```

```
</html>
```

Note the symbol for
line continuation

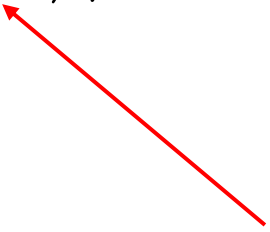


JavaScript Statements

```
<html>  
<head><title>My Page</title></head>  
<body>  
<script language="JavaScript">
```

```
document.write('<b1>This is my first →  
JavaScript Page</b1>');
```

```
</script>  
</body>  
</html>
```



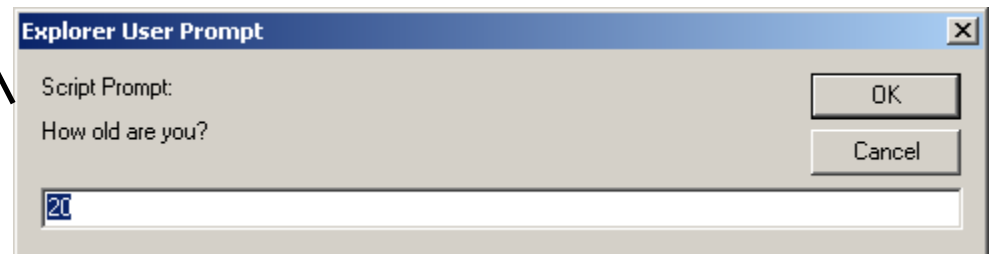
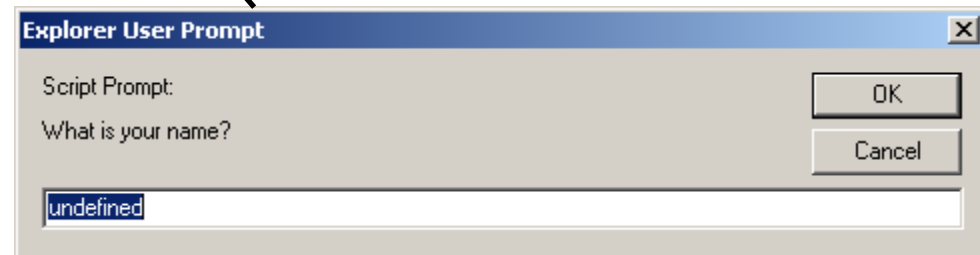
**HTML written
inside JavaScript**

Hiding JavaScript from Incompatible Browsers

```
<script type="text/javascript">  
  <!--  
    document.writeln("Hello, WWW");  
  // -->  
</script>  
<noscript>  
  Your browser does not support JavaScript.  
</noscript>
```

alert(), confirm(), and prompt()

```
<script type="text/javascript">  
alert("This is an Alert method");  
confirm("Are you OK?");  
prompt("What is your name?");  
prompt("How old are you?", "20");  
</script>
```



alert() and confirm()

```
alert("Text to be displayed");
```

- Display a message in a dialog box.
- The dialog box will block the browser.

```
var answer = confirm("Are you sure?");
```

- Display a message in a dialog box with two buttons: "OK" or "Cancel".
- **confirm()** returns **true** if the user click "OK". Otherwise it returns **false**.

prompt()

```
prompt("What is your student id number?");  
prompt("What is your name?", "No name");
```

- Display a message and allow the user to enter a value
- The second argument is the "default value" to be displayed in the input textfield.
- Without the default value, "undefined" is shown in the input textfield.
- If the user click the "OK" button, **prompt()** returns the value in the input textfield as a string.
- If the user click the "Cancel" button, **prompt()** returns null.

window.alert

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0  
Transitional//EN">  
<HTML>  
<HEAD>  
<TITLE> Using window.alert </TITLE>  
<SCRIPT TYPE="text/javascript">  
    window.alert( "Welcome to\nJavaScript\nProgramming!" );  
</SCRIPT>  
</HEAD>  
<BODY>  
<P>Click Refresh (or Reload) to run this script again.</P>  
</BODY>  
</HTML>
```

User input/output

```
<SCRIPT TYPE="text/javascript">  
    var firstNumber, // first string entered by user  
        secondNumber, // second string entered by user  
        number1,      // first number to add  
        number2,      // second number to add  
        sum;          // sum of number1 and number2  
    // read in first number from user as a string  
    firstNumber = window.prompt("Enter first integer", "0" );  
    // read in second number from user as a string  
    secondNumber = window.prompt( "Enter second integer", "0" );  
    // convert numbers from strings to integers  
    firstNumber = parseInt(firstNumber);  
    number2 = parseInt( secondNumber );  
    // add the numbers  
    sum = firstNumber + number2;  
    // display the results  
    document.writeln( "<H1>The sum is " + sum + "</H1>" );  
</SCRIPT>
```

Creating JavaScript Functions

- Function names are case-sensitive.
- The function name must begin with a letter or underscore (_) and cannot contain any spaces.
- There is no limit to the number of function parameters that a function may contain.
- The parameters must be placed within parentheses, following the function name, and the parameters must be separated by commas.

Creating JavaScript Functions

```
function function_name(parameters) {  
    JavaScript commands  
}
```

- *parameters* are the values sent to the function
(note: not all functions require parameters)
- { and } are used to mark the beginning and end of the commands in the function.

Functions

```
<SCRIPT TYPE = "text/javascript">
    var input1 = window.prompt( "Enter first number", "0" );
    var input2 = window.prompt( "Enter second number", "0"
);
    var input3 = window.prompt( "Enter third number", "0" );
    var value1 = parseFloat( input1 );
    var value2 = parseFloat( input2 );
    var value3 = parseFloat( input3 );
    var maxValue = maximum( value1, value2, value3 );
    document.writeln( "First number: " + value1 +
"<BR>Second number: " + value2 +
    "<BR>Third number: " + value3 +
"<BR>Maximum is: " + maxValue );
    // maximum method definition (called from above)
    function maximum( x, y, z ) {
        return Math.max( x, Math.max( y, z ) );
    }
</SCRIPT>
```

Random Numbers

```
<SCRIPT TYPE="text/javascript">
    var value;
    document.writeln( "<H1>Random Numbers</H1>" +
        "<TABLE BORDER = '1' WIDTH = '50%'><TR>" );
    for ( var i = 1; i <= 20; i++ ) {
        value = Math.floor( 1 + Math.random() * 6 );
        document.writeln( "<TD>" + value + "</TD>" );
        if ( i % 5 == 0 && i != 20 )
            document.writeln( "</TR><TR>" );
    }
    document.writeln( "</TR></TABLE>" );
</SCRIPT>
```

Identifier

- Same as Java/C++ except that it allows an additional character – '\$'.
- Contains only 'A' – 'Z', 'a' – 'z', '0' – '9', '_', '\$'
- First character cannot be a digit
- Case-sensitive
- Cannot be reserved words or keywords

Variable and Variable Declaration

```
<head><script type="text/javascript">
  // We are in the default scope - the "window" object.
  x = 3;          // same as "window.x = 3"
  var y = 4;      // same as "y = 4" or "window.y = 4"

  { // Introduce a block to create a local scope
    x = 0;        // Same as "window.x = 0"
    var y = 1;    // This is a local variable y
  }

  alert("x=" + x + ", y=" + y); // Print x=0, y=4

</script></head>
```

- Local variable is declared using the keyword 'var'.
- Dynamic binding – a variable can hold any type of value
- If a variable is used without being declared, the variable is created automatically.
 - If you misspell a variable name, program will still run (but works incorrectly)

Data Types

- Primitive data types
 - Number: integer & floating-point numbers
 - Boolean: true or false
 - String: a sequence of alphanumeric characters
- Composite data types (or Complex data types)
 - Object: a named collection of data
 - Array: a sequence of values (an array is actually a predefined object)
- Special data types
 - Null: the only value is "null" – to represent nothing.
 - Undefined: the only value is "undefined" – to represent the value of an uninitialized variable

Strings

- A string variable can store a sequence of alphanumeric characters, spaces and special characters.
- Each character is represented using 16 bit
 - You can store Chinese characters in a string.
- A string can be enclosed by a pair of single quotes (') or double quote (").
- Use escaped character sequence to represent special character (e.g.: `\", \n, \t`)

typeof operator

```
var x = "hello", y;  
alert("Variable x value is " + typeof x );  
alert("Variable y value is " + typeof y );  
alert("Variable x value is " + typeof z );
```

- An unary operator that tells the type of its operand.
 - Returns a string which can be "number", "string", "boolean", "object", "function", "undefined", and "null"
 - An array is internally represented as an object.

Object

- An object is a collection of **properties**.
- Properties can be variables (Fields) or Functions (Methods)
- There is no "Class" in JavaScript.

Using Arrays

- An **array** is an ordered collection of values referenced by a single variable name.
- The syntax for creating an array variable is:
var *variable* = new Array(*size*);
 - *variable* is the name of the array variable
 - *size* is the number of elements in the array (optional)
- To populate the array with values, use:
***variable*[*i*]=*value*;**
where *i* is the *i*th item of the array. The 1st item has an index value of **0**.

Array

- An array is represented by the **Array** object. To create an array of N elements, you can write

```
var myArray = new Array(N) ;
```

- Index of array runs from 0 to N-1.
- Can store values of different types
- Property "**length**" tells the # of elements in the array.
- Consists of various methods to manipulate its elements. e.g., **reverse()**, **push()**, **concat()**, etc

Array Examples

```
var Car = new Array(3);  
Car[0] = "Ford";  
Car[1] = "Toyota";  
Car[2] = "Honda";
```

```
// Create an array of three elements with initial  
// values
```

```
var Car2 = new Array("Ford", "Toyota", "Honda");
```

```
// Create an array of three elements with initial  
// values
```

```
var Car3 = ["Ford", "Toyota", "Honda"];
```

```
// An array of 3 elements, each element is undefined
var tmp1 = new Array(3);

// An array of 3 elements with initial values
var tmp2 = new Array(10, 100, -3);

// An array of 3 elements with initial values
// of different types
var tmp3 = new Array(1, "a", true);

// Makes tmp3 an array of 10 elements
tmp3.length = 10; // tmp[3] to tmp[9] are undefined.

// Makes tmp3 an array of 100 elements
tmp3[99] = "Something";
// tmp[3] to tmp[98] are undefined.
```

Using Arrays

To create and populate the array in a single statement, use:

var variable = new Array(values);

- ***values*** are the array elements enclosed in quotes and separated by commas
- ***var MonthTxt=new Array("", "January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December");***
 - January will have an index value of "1".

Null & Undefined

- An undefined value is represented by the keyword "**undefined**".
 - It represents the value of an uninitialized variable
- The keyword "**null**" is used to represent “nothing”
 - Declare and define a variable as “null” if you want the variable to hold nothing.
 - Avoid leaving a variable undefined.

Working with Expressions and Operators

- Expressions are JavaScript commands that assign values to variables.
- Expressions are created using variables, values, and operators.
- The + operator performs the action of adding or combining two elements. For example,
 - `var ThisMonth = Today.getMonth()+1;`

OPERATOR	DESCRIPTION	EXAMPLE
+	Adds two values together	var Men = 20; var Women = 25; var TotalPeople = Men + Women;
-	Subtracts one value from another	var Price = 1000; var Expense = 750; var Profit = Price - Expense;
*	Multiplies two values together	var Width = 50; var Length = 25; var Area = Width*Length;
/	Divides one value by another	var People = 50; var TotalCost = 200; var CostperPerson = TotalCost/People;
%	Shows the remainder after dividing one value by another	var TotalEggs = 64; var CartonSize = 12; var EggsLeft = TotalEggs % CartonSize;
++	Increases a value by 1 (unary operator)	var Eggs = 12; var BakersDozen = Eggs++;
--	Decreases a value by 1 (unary operator)	var Eggs = 12; var EggsIfOnesBroken = Eggs--;
-	Changes the sign of a value (unary operator)	var MyGain = 50; var YourLoss = - MyGain;

Operators

- Binary operators work on two elements in an expression.
- Unary operators work on only one variable.
 - unary operators include: the increment (++), decrement (--), and negation (-) operators.
- An increment operator is used to increase the value of the x variable by one.

```
x = 100;
```

```
y = x++;
```

Operators

- The decrement operator reduces the value of a variable by 1.

```
x = 100;
```

```
y = x--;
```

- The negation operator changes the sign of a variable:

```
x = -100;
```

```
y = -x;
```

Assignment Operators

- Expressions assign values using assignment operators. “=” is the most common one.
- Additional includes the += operator
- The following create the same results:
 $x = x + y;$
 $x += y$
- Either of the following increase the value of the x variable by 2:
 $x = x + 2;$
 $x += 2$

Assignment Operators

OPERATOR	DESCRIPTION
=	Assigns the value of the variable on the right to the variable on the left ($x = y$)
+=	Adds the two variables and assigns the result to the variable on the left (equivalent to $x = x + y$)
-=	Subtracts the variable on the right from the variable on the left and assigns the result to the variable on the left (equivalent to $x = x - y$)
*=	Multiplies the two variables together and assigns the result to the variable on the left (equivalent to $x = x * y$)
/=	Divides the variable on the left by the variable on the right and assigns the result to the variable on the left (equivalent to $x = x / y$)
%=	Divides the variable on the left by the variable on the right and assigns the remainder to the variable on the left (equivalent to $x = x \% y$)

Comparison, Logical, and Conditional Operators

To create a condition, you need one of three types of operators:

- a **comparison operator** compares the value of one element with that of another, which creates a **Boolean expression** that is either true or false
- a **logical operator** connects two or more Boolean expressions
- a **conditional operator** tests whether a specific condition is true and returns one value if the condition is true and a different value if the condition is false

An Example of Boolean Expressions

- **`x < 100;`**
 - if **`x`** is less than 100, this expression returns the value true; however, if **`x`** is 100 or greater, the expression is false
- **`y == 20;`**
 - the **`y`** variable must have an exact value of 20 for the expression to be true
 - comparison operator uses a double equal sign (**`==`**)

Comparison Operators

OPERATOR	DESCRIPTION
<code>==</code>	Returns true if variables are equal ($x = y$)
<code>!=</code>	Returns true if variables are not equal ($x \neq y$)
<code>></code>	Returns true if the variable on the left is greater than the variable on the right ($x > y$)
<code><</code>	Returns true if the variable on the left is less than the variable on the right ($x < y$)
<code>>=</code>	Returns true if the variable on the left is greater than or equal to the variable on the right ($x \geq y$)
<code><=</code>	Returns true if the variable on the left is less than or equal to the variable on the right ($x \leq y$)

A Logical Operator

- The logical operator **&&** returns a value of true only if all of the Boolean expressions are true.

OPERATOR	DESCRIPTION	EXAMPLE
In the following examples, assume that x = 20 y = 25		
&&	Returns true when both expressions are true.	(x == 20) && (y == 25) returns true (x == 20) && (y == 20) returns false
	Returns true when either expression is true.	(x == 20) (y == 20) returns true (x == 25) (y == 20) returns false
!	Returns true if the expression is false and false if the expression is true.	! (x == 20) returns false ! (x == 25) returns true

A Conditional Operator

tests whether a specific condition is true and returns one value if the condition is true and a different value if the condition is false.

- **Message = (mail == “Yes”) ? “You have mail”: “No mail”;**
- tests whether the **mail** variable is equal to the value “Yes”
 - if it is, the **Message** variable has the value “You have mail”;
 - otherwise, the **Message** variable has the value “No mail”.

Working with Conditional Statements

```
if (condition) {  
    JavaScript Commands  
}
```

- ***condition*** is an expression that is either true or false
- if the condition is true, the *JavaScript Commands* in the command block are executed
- if the condition is not true, then no action is taken

Using an if...else Statement

```
if (condition) {  
    JavaScript Commands if true  
} else  
    JavaScript Commands if false  
}
```

- *condition* is an expression that is either true or false, and one set of commands is run if the expression is true, and another is run if the expression is false

if...else Conditional Statement

```
document.write("Today is " + ThisMonth +  
    "/" + ThisDay + "/" + ThisYear + "<br />");  
if (DaysLeft > 0) {  
    document.write("Only " + DaysLeft +  
        " days until Christmas");  
} else {  
    document.write("Happy Holidays from  
        North Pole Novelties");  
}
```

Using an if...else if Statement

```
if (condition) {  
    JavaScript Commands if true  
} else if (cond)  
    JavaScript Commands if false  
}  
else  
{
```

- *condition* is an expression that is either true or false, and one set of commands is run if the expression is true, and another is run if the expression is false

Using switch Statement

```
switch (expr) {  
    case val:...  
        break;  
    default: stmts;
```

Working with Program Loops

- A **program loop** is a set of instructions that is executed repeatedly.
- There are two types of loops:
 - loops that repeat a set number of times before quitting
 - loops that repeat as long as a certain condition is met

The For Loop

- The **For loop** allows you to create a group of commands to be executed a set number of times through the use of a **counter** that tracks the number of times the command block has been run.
- Set an initial value for the counter, and each time the command block is executed, the counter changes in value.
- When the counter reaches a value above or below a certain stopping value, the loop ends.

The For Loop Continued

```
for (start; condition; update) {
```

JavaScript Commands

```
}
```

- ***start*** is the starting value of the counter
- ***condition*** is a Boolean expression that must be true for the loop to continue
- ***update*** specifies how the counter changes in value each time the command block is executed

```
<table border>
<tr>
<script>
    for (num = 1; num <=4; num++) {
        document.write("<td>" + num + "</td>");
    }
</script>
</tr>
</table>
```

For loop

1	2	3	4
---	---	---	---

resulting table

```
<table border>
<script>
for (rownum = 1; rownum <=3; rownum++) {
    document.write("<tr>");
    for (colnum = 1; colnum <=4; colnum++) {
        document.write("<td>" + rownum + "," + colnum + "</td>");
    }
    document.write("</tr>");
}
</script>
</table>
```

nested For loop

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4

resulting table

Specifying Counter Values in a For Loop

FOR LOOP	COUNTER VALUES
for (i = 1; i <= 5; i++)	i = 1, 2, 3, 4, 5
for (i = 5; i > 0; i--)	i = 5, 4, 3, 2, 1
for (i = 0; i <= 360; i += 60)	i = 0, 60, 120, 180, 240, 300, 360
for (i = 2; i <= 64; i *= 2)	i = 2, 4, 8, 16, 32, 64

“for/in” statement

```
for (var variable in object) {  
    statements;  
}
```

- To iterate through all the properties in "object".
- "**variable**" takes the name of each property in "object"
- Can be used to iterate all the elements in an Array object.

```
var keys = "", values = "";
var mylist = new Array("Chinese", "English", "Jap");
mylist.newField1 = "Something";

for (var key in booklist) {
    keys += key + " ";
    values += booklist[counter] + " ";
}

// keys becomes "0 1 2 newField1"
// values becomes "Chinese English Jap Something"
```

The While Loop

- The **While loop** runs a command group as long as a specific condition is met, but it does not employ any counters.

- The general syntax of the While loop is:

```
while (condition) {  
    JavaScript Commands  
}
```

- ***condition*** is a Boolean expression that can be either true or false

```
do{  
  Stmts;  
}  
while (condition)
```

```
<table border>
<tr>
<script>
  var num = 1;
  while (num <= 4) {
    document.write("<td>" + num + "</td>");
    num++;
  }
</script>
</tr>
</table>
```

While loop

1	2	3	4
---	---	---	---

resulting table

```
<table border>
<script>
var rownum = 1;
var colnum = 1;
while (rownum <=3) {
    document.write("<tr>");
    while (colnum <=4) {
        document.write("<td>" + rownum + "," + colnum + "</td>");
        colnum++;
    }
    document.write("</tr>");
    colnum = 1;
    rownum++;
}
</script>
</table>
```

nested While loop

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4

resulting table

Built-In Functions

- **eval (expr)**

- evaluates an expression or statement
 - `eval("3 + 4");` // Returns 7 (Number)
 - `eval("alert('Hello')");` // Calls the function `alert('Hello')`

- **isFinite (x)**

- Determines if a number is finite

- **isNaN (x)**

- Determines whether a value is “Not a Number”

Built-In Functions

- **parseInt(s)**
- **parseInt(s, radix)**
 - Converts string literals to integers
 - Parses up to any character that is not part of a valid integer
 - `parseInt("3 chances")` // returns 3
 - `parseInt(" 5 alive")` // returns 5
 - `parseInt("How are you")` // returns NaN
 - `parseInt("17", 8)` // returns 15
- **parseFloat(s)**
 - Finds a floating-point value at the beginning of a string.
 - `parseFloat("3e-1 xyz")` // returns 0.3
 - `parseFloat("13.5 abc")` // returns 13.5

Creating Objects

- JavaScript is not an OOP language.
- "prototype" is the closest thing to "class" in JavaScript.
- Next few slides show several ways to create objects
- It is also possible to emulate "inheritance" in JavaScript.

Creating objects using `new Object()`

```
var person = new Object();

// Assign fields to object "person"
person.firstName = "John";
person.lastName = "Doe";

// Assign a method to object "person"
person.sayHi = function() {
    alert("Hi! " + this.firstName + " " + this.lastName);
}

person.sayHi(); // Call the method in "person"
```

Creating objects using Literal Notation

```
var person = {  
    // Declare fields  
    // (Note: Use comma to separate fields)  
    firstName : "John",  
    lastName  : "Doe",  
  
    // Assign a method to object "person"  
    sayHi : function() {  
        alert("Hi! " + this.firstName + " " +  
            this.lastName);  
    }  
}  
  
person.sayHi(); // Call the method in "person"
```

Creating objects using Literal Notation (Nested notation is possible)

```
var triangle = {  
    // Declare fields (each as an object of two fields)  
    p1 : { x : 0, y : 3 },  
    p2 : { x : 1, y : 4 },  
    p3 : { x : 2, y : 5 }  
}  
  
alert(triangle.p1.y);    // Show 3
```

Object Constructor and prototyping

```
function Person(fname, lname) {  
    // Define and initialize fields  
    this.firstName = fname;  
    this.lastName = lname;  
  
    // Define a method  
    this.sayHi = function() {  
        alert("Hi! " + this.firstName + " " +  
            this.lastName);  
    }  
}  
  
var p1 = new Person("John", "Doe");  
var p2 = new Person("Jane", "Dow");  
  
p1.sayHi();    // Show "Hi! John Doe"  
p2.sayHi();    // Show "Hi! Jane Dow"
```

Adding methods to objects using prototype

```
// Suppose we have defined the constructor "Person"
// (as in the previous slide).

var p1 = new Person("John", "Doe");
var p2 = new Person("Jane", "Dow");

// Attaching a new method to all instances of Person
Person.prototype.sayHello = function() {
    alert("Hello! " + this.firstName + " " +
          this.lastName);
}

// We can also introduce new fields via "prototype"

p1.sayHello(); // Show "Hello! John Doe"
p2.sayHello(); // Show "Hello! Jane Dow"
```

Events

- An event occurs as a result of some activity
 - e.g.:
 - A user clicks on a link in a page
 - Page finished loaded
 - Mouse cursor enter an area
 - A preset amount of time elapses
 - A form is being submitted

Event Handlers

- **Event Handler** – a segment of codes (usually a function) to be executed when an event occurs
- We can specify event handlers as attributes in the HTML tags.
- The attribute names typically take the form "**onXXX**" where **XXX** is the event name.
 - e.g.:
`<a href="..." onClick="alert('Bye') ">Other Website</a>`

Event Handlers

Event Handlers	Triggered when
onChange	The value of the text field, textarea, or a drop down list is modified
onClick	A link, an image or a form element is clicked once
onDbClick	The element is double-clicked
onMouseDown	The user presses the mouse button
onLoad	A document or an image is loaded
onSubmit	A user submits a form
onReset	The form is reset
onUnLoad	The user closes a document or a frame
onResize	A form is resized by the user

onClick Event Handler Example

```
<html>
<head>
<title>onClick Event Handler Example</title>
<script type="text/javascript">
function warnUser() {
    return confirm("Are you a student?");
}
</script>
</head>
<body>
<a href="ref.html" onClick="return warnUser()">
<!--
    If onClick event handler returns false, the link
    is not followed.
-->
Students access only</a>
</body>
</html>
```

onLoad Event Handler Example

```
<html><head>
<title>onLoad and onUnload Event Handler Example</title>
</head>
<body
  onLoad="alert('Welcome to this page')"
  onUnload="alert('Thanks for visiting this page')"
>
Load and UnLoad event test.
</body>
</html>
```

onMouseOver & onMouseOut Event Handler

```
<html>
<head>
<title>onMouseOver / onMouseOut Event Handler Demo</title>
</head>
<body>
<a href="http://www.cuhk.edu.hk"
  onMouseOver="window.status='CUHK Home'; return true;"
  onMouseOut="status=' '"
>CUHK</a>
</body>
</html>
```

- When the mouse cursor is over the link, the browser displays the text "CUHK Home" instead of the URL.
- The "return true;" of **onMouseOver** forces browser not to display the URL.
- window.status and window.defaultStatus are disabled in Firefox.

onSubmit Event Handler Example

```
<html><head>
<title>onSubmit Event Handler Example</title>
<script type="text/javascript">
    function validate() {
        // If everything is ok, return true
        // Otherwise return false
    }
</script>
</head>
<body>
<form action="MessageBoard" method="POST"
    onSubmit="return validate();"
>
...
</form></body></html>
```

- If `onSubmit` event handler returns false, data is not submitted.
- If `onReset` event handler returns false, form is not reset

Build-In JavaScript Objects

Object	Description
Array	Creates new array objects
Boolean	Creates new Boolean objects
Date	Retrieves and manipulates dates and times
Error	Returns run-time error information
Function	Creates new function objects
Math	Contains methods and properties for performing mathematical calculations
Number	Contains methods and properties for manipulating numbers.
String	Contains methods and properties for manipulating text strings

String Object (Some useful methods)

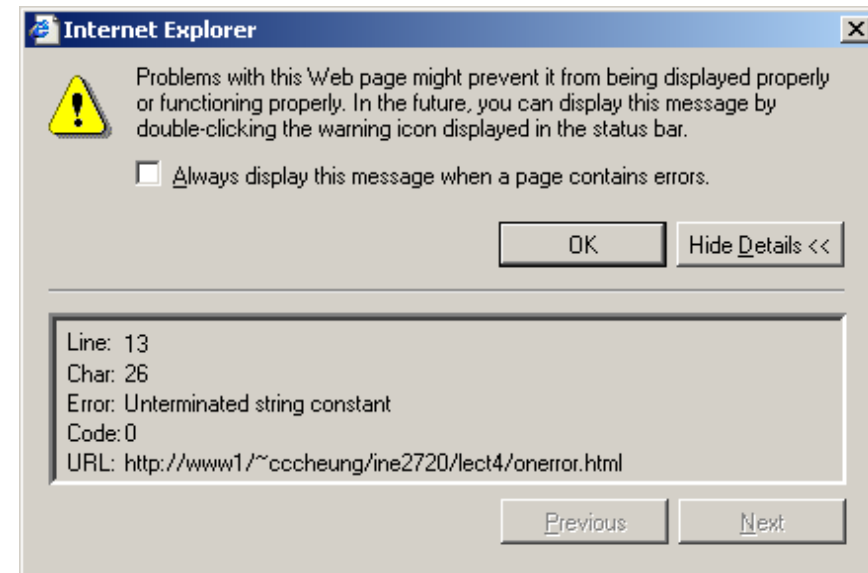
- `length`
 - A string property that tells the number of character in the string
- `charAt(idx)`
 - Returns the character at location "idx"
- `toUpperCase(), toLowerCase()`
 - Returns the same string with all uppercase/lowercase letters
- `substring(beginIdx)`
 - Returns a substring started at location "beginIdx"
- `substring(beginIdx, endIdx)`
 - Returns a substring started at "beginIdx" until "endIdx" (but not including "endIdx")
- `indexOf(str)`
 - Returns the position where "str" first occurs in the string

Error and Exception Handling in JavaScript

- Javascript makes no distinction between Error and Exception (Unlike Java)
- Handling Exceptions
 - The **onError** event handler
 - A method associated with the window object.
 - It is called whenever an exception occurs
 - The **try ... catch ... finally** block
 - Similar to Java try ... catch ... finally block
 - For handling exceptions in a code segment
 - Use **throw** statement to throw an exception
 - You can throw value of any type
 - The **Error** object
 - Default object for representing an exception
 - Each Error object has a **name** and **message** properties

How to use “onError” event handler?

```
<html>
<head>
<title>onerror event handler example</title>
<script type="text/javascript">
function errorHandler() {
    alert("Error Ourred!");
}
// JavaScript is casesensitive
// Don't write onerror!
window.onError = errorHandler;
</script>
</head>
<body>
<script type="text/javascript">
    document.write("Hello there;
</script>
</body>
</html>
```



try ... catch ... finally

```
try {  
    // Contains normal codes that might throw an exception.  
  
    // If an exception is thrown, immediately go to  
    //    catch block.  
  
} catch ( errorVariable ) {  
    // Codes here get executed if an exception is thrown  
    //    in the try block.  
  
    // The errorVariable is an Error object.  
  
} finally {  
    // Executed after the catch or try block finish  
  
    // Codes in finally block are always executed  
}  
// One or both of catch and finally blocks must accompany the try  
// block.
```

try ... catch ... finally example

```
<script type="text/javascript">
try{
    document.write("Try block begins<br>");
    // create a syntax error
    eval ("10 + * 5");

} catch( errVar ) {
    document.write("Exception caught<br>");
    // errVar is an Error object
    // All Error objects have a name and message properties
    document.write("Error name: " + errVar.name + "<br>");
    document.write("Error message: " + errVar.message +
                    "<br>");
} finally {
    document.write("Finally block reached!");
}
</script>
```

Throwing Exception

```
<script type="text/javascript">
try{
  var num = prompt("Enter a number (1-2):", "1");
  // You can throw exception of any type
  if (num == "1")
    throw "Some Error Message";
  else
    if (num == "2")
      throw 123;
    else
      throw new Error ("Invalid input");
} catch( err ) {
  alert(typeof(errMsg) + "\n" + err);
  // instanceof operator checks if err is an Error object
  if (err instanceof Error)
    alert("Error Message: " + err.message);
}
</script>
```